



DESIGN PATTERNS

01 - INTRODUKTION

Oversigt

- ▶ Onsdag 09.04.25 - intro, command, singleton og component
- ▶ Torsdag 10.04.25 - Creational Patterns (og strategy) med KERA
- ▶ Onsdag 11.04.25 - Builder, observer og animator
- ▶ Mandag 14.04.25 - Collision, loading og State – Projekt opstart med ANVP
- ▶ mandag 28.04.24 - projekt aflevering

Plan

- ▶ Videor
 - ▶ i kan tage i eget tempo.
 - ▶ Dagsinddelt
 - ▶ Forventes stadig at i møder op.
 - ▶ Underviser på, mulighed for spørgsmål under hele forløbet.
 - ▶ Kera på den 10/04

FORVENTNINGER

▶ **DET FORVENTES AT**

- ▶ I har en god forståelse for MonoGame frameworket
- ▶ I kan læse et klassediagram og implementere det i C#

▶ **DENNE UGE og næste**

- ▶ Der arbejdes videre med det samme projekt hele faget.
- ▶ Derfor er det vigtigt at I laver alle øvelserne.
 - ▶ Og i den rækkefølge de er givet i!



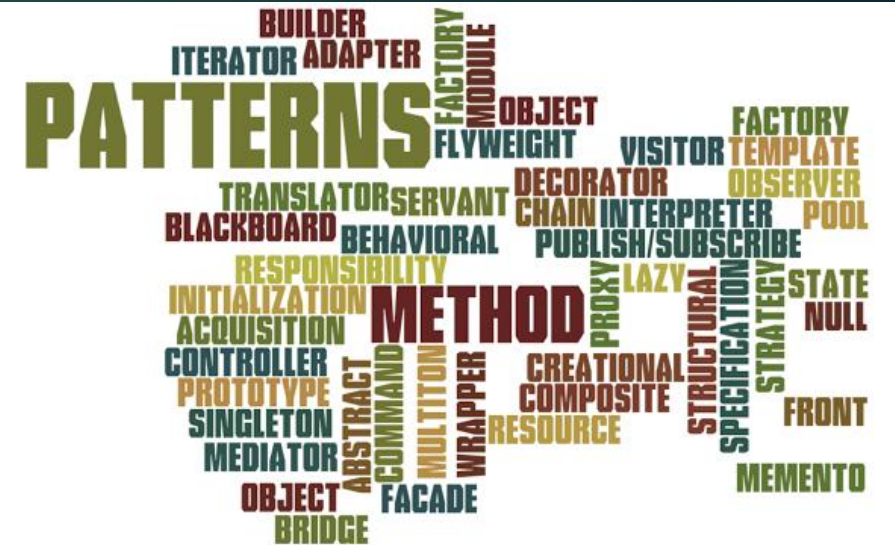
DESIGNPATTERNS HISTORISK?

- ▶ 1970'erne-1980'erne:
 - ▶ Begyndelsen af objektorienteret programmering (OOP) og komplekse systemer.
- ▶ 1994:
 - ▶ Design Patterns: Elements of Reusable Object-Oriented Software af Erich Gamma, Richard Helm, Ralph Johnson og John Vlissides (Gang of Four - GoF).
- ▶ Bidrag:
 - ▶ Etablering af 23 klassiske designmønstre.

HVAD ER DESIGNPATTERNS?

- ▶ Hvad er en design pattern?
 - ▶ En løsning til en generel problemstilling
 - ▶ Det er en beskrivelse/template for hvordan et problem kan løses i forskellige situationer
 - ▶ Et design pattern viser typisk forholdet og interaktionen mellem klasserne, uden at specificere den endelige applikation den skal inkluderes i.
- ▶ Vi må meget gerne lave vores egne tilføjelser til et pattern. Det er meget sjældent at et pattern kan tages i brug uden tilpasninger.
 - ▶ F.eks. Kan det være at vi vil benytte en abstrakt klasse i stedet for et interface.
 - ▶ Eller vi kan have brug for at tilføje en constructor eller lignende.

- [illegible]



DESIGN PATTERNS

- ▶ Hvad består et design pattern af?
 - ▶ Navn: Bruges til at genkende løsningen
 - ▶ Problem: Beskriver hvornår et pattern skal bruges
 - ▶ Løsning: Beskriver forholdet mellem elementerne, som indgår i et design
 - ▶ Konsekvenser: Resultatet og "trade-offs" der forekommer når et pattern tilføjes



DESIGN PATTERNS

Der findes forskellige typer af design patterns

- ▶ Structural

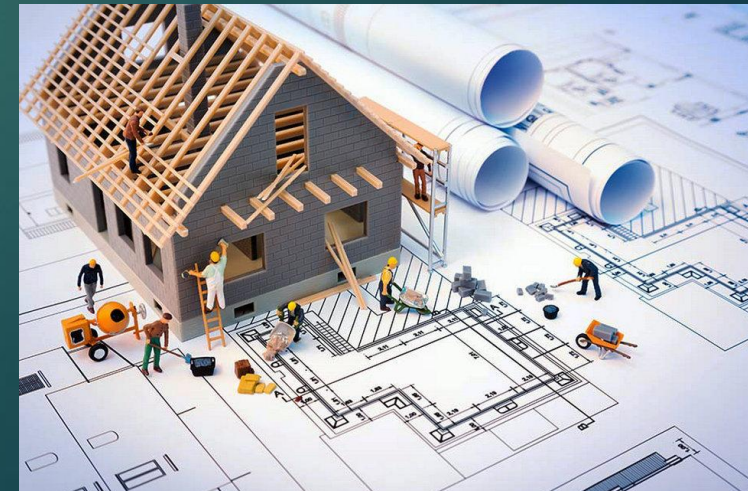
- ▶ Håndterer forholdet mellem objekter, gør det lettere for objekter at arbejde sammen

- ▶ Creational

- ▶ Håndterer skabelsen af objekter, gør det lettere at skabe objekter på en måde der passer til den givende situation.

- ▶ Behavioural

- ▶ Bruges til at gøre kommunikationen mellem objekter lettere og mere fleksibel



DESIGN PATTERNS

De originale GOF patterns er kategoriseret således:

▶ Creational Patterns:

- ▶ Singleton
- ▶ Factory Method
- ▶ Abstract Factory
- ▶ Builder
- ▶ Prototype

▶ Structural Patterns:

- ▶ Adapter
- ▶ Bridge
- ▶ Composite
- ▶ Decorator
- ▶ Facade
- ▶ Flyweight
- ▶ Proxy

▶ Behavioral Patterns:

- ▶ Chain of Responsibility
- ▶ Command
- ▶ Interpreter
- ▶ Iterator
- ▶ Mediator
- ▶ Memento
- ▶ Observer
- ▶ State
- ▶ Strategy
- ▶ Template Method
- ▶ Visitor

DESIGN PATTERNS

Af disse kommer vi til at kigge nærmere på disse:

▶ Creational Patterns:

- ▶ Singleton
- ▶ Factory Method
- ▶ Abstract Factory
- ▶ Builder
- ▶ Prototype
- ▶ + Lazy og Eager loading

▶ Structural Patterns:

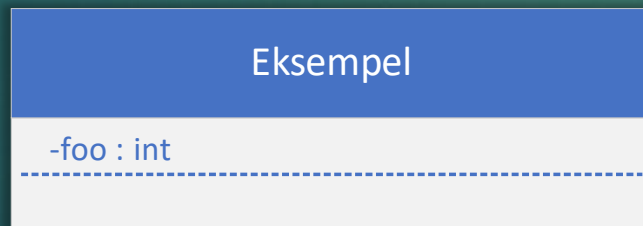
- ▶ ~~Adapter~~
- ▶ ~~Bridge~~
- ▶ Composite
- ▶ ~~Decorator~~
- ▶ ~~Facade~~
- ▶ ~~Flyweight~~
- ▶ ~~Proxy~~

▶ Behavioral Patterns:

- ▶ ~~Chain of Responsibility~~
- ▶ Command
- ▶ ~~Interpreter~~
- ▶ ~~Iterator~~
- ▶ ~~Mediator~~
- ▶ ~~Memento~~
- ▶ Observer
- ▶ State
- ▶ Strategy
- ▶ ~~Template Method~~
- ▶ ~~Visitor~~

INFO

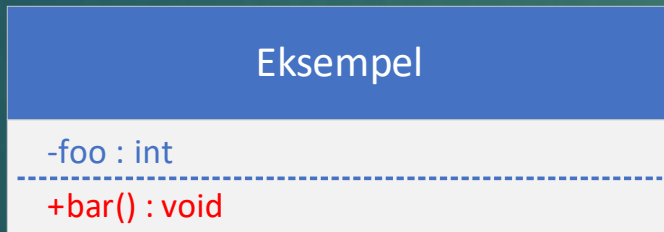
- ▶ Hvordan læser vi klassesdiagrammer
 - ▶ Første gang vi ser diagrammet:
 - ▶ Betyder at hele klassen skal implementeres som vist.



```
public class Eksempel
{
    private int foo;
}
```

INFO

- ▶ Hvordan læser vi klassediagrammer
 - ▶ Næste gang vi møder diagrammet ser det ud på følgende måde:
 - ▶ Betyder at metoden "bar" skal tilføjes til klassen.

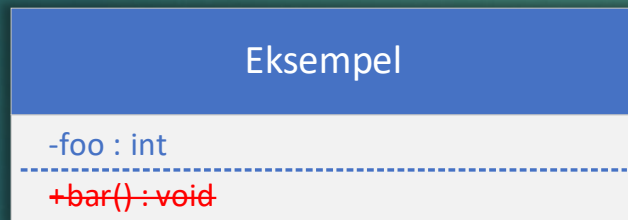


```
public class Eksempel
{
    private int foo;

    0 references
    public void Bar()
    {
        ...
    }
}
```

INFO

- ▶ Næste gang vi møder diagrammet ser det ud på følgende måde:
 - ▶ Betyder at metoden "bar" skal fjernes fra klassen.

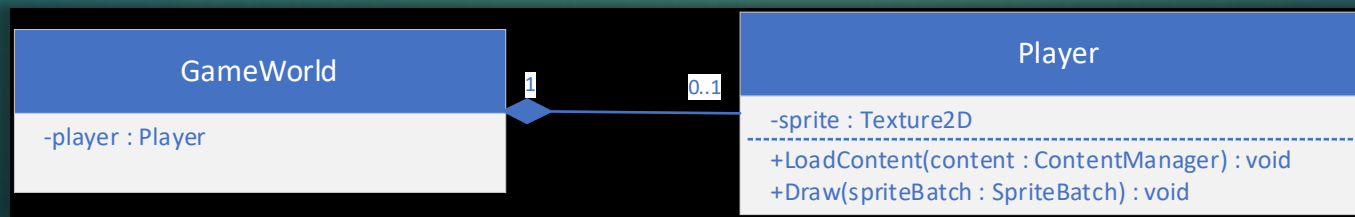


```
public class Eksempel
{
    private int foo;
}
```

ØVELSE 1

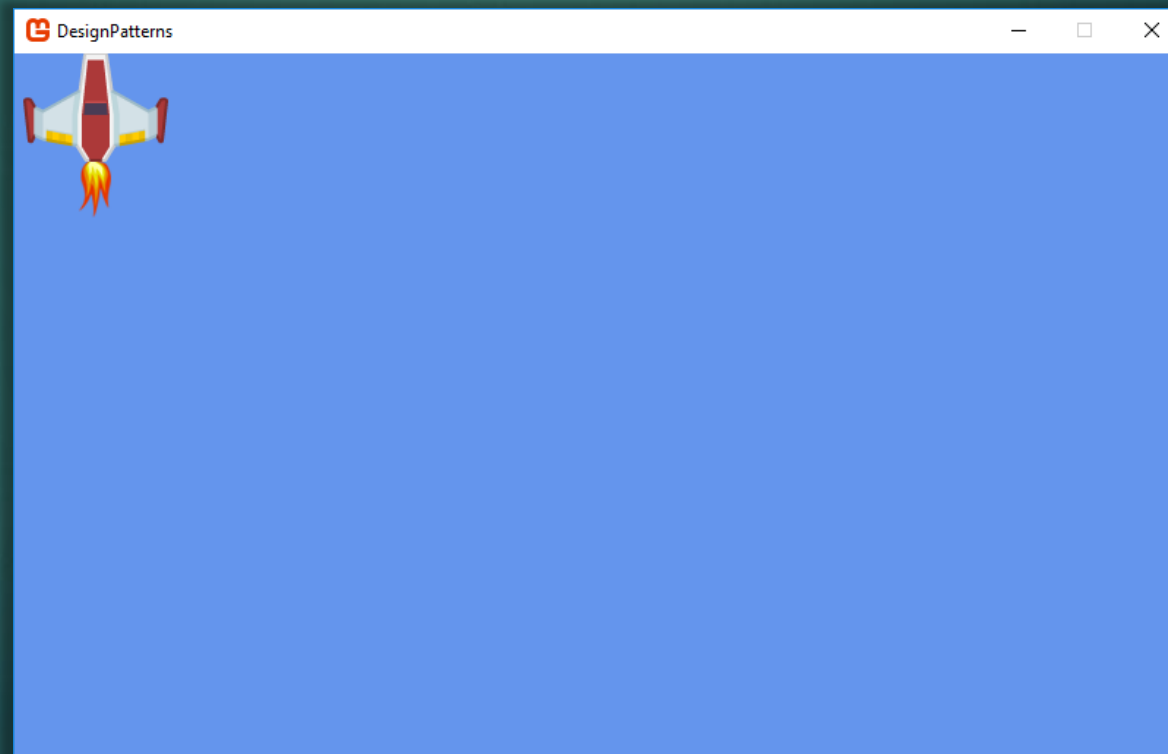
Vi skal arbejde på det samme projekt hele ugen.

- ▶ De sprites jeg bruger kan findes på Moodle
- ▶ Implementer følgende klassediagram i MonoGame
- ▶ Det forventes at I har forståelse for hvordan metoder som Draw og LoadContent skal implementeres og kaldes.
 - ▶ Hvis man ikke er sikker spørger man selvfølgelig bare 😊



ØVELSE 1

- ▶ Det forventes at I har følgende når diagrammet er implementeret



Undervisning:

<https://www.youtube.com/watch?v=YAKkju-rY7Q>

COMMAND PATTERN

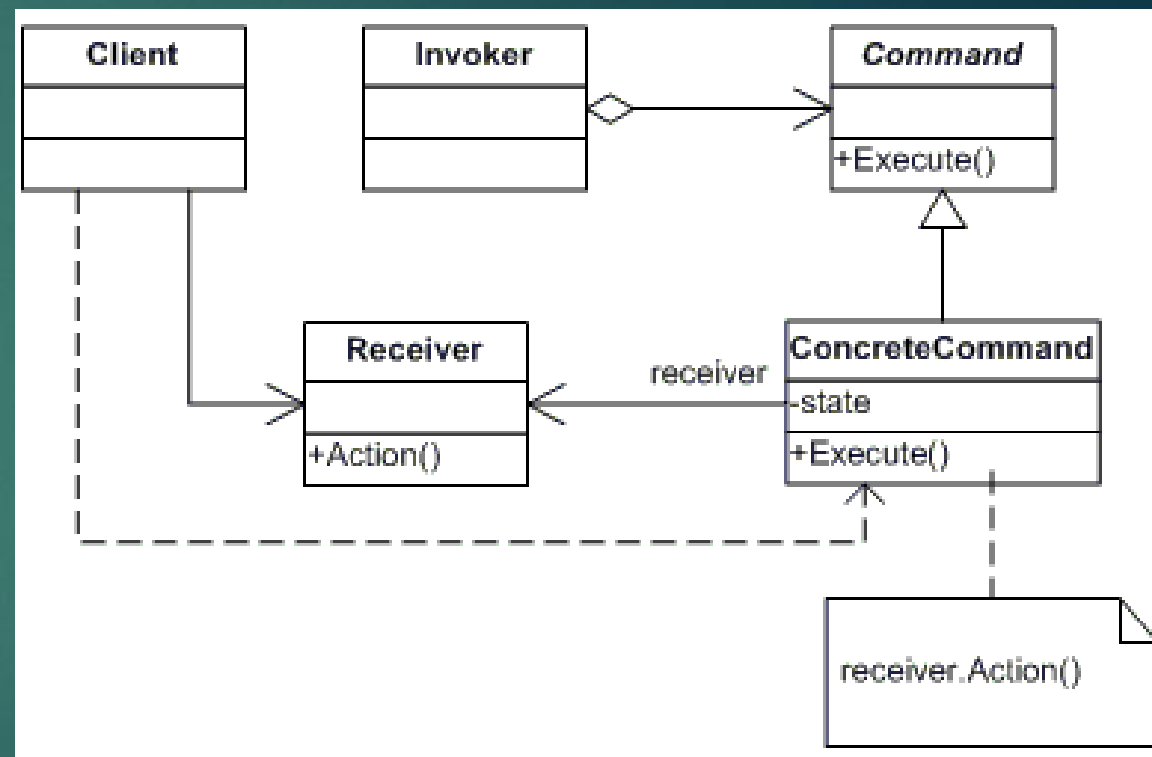
*“ENCAPSULATE A REQUEST AS AN OBJECT, THEREBY LETTING USERS
PARAMETERIZE CLIENTS WITH DIFFERENT REQUESTS, QUEUE OR LOG
REQUESTS, AND SUPPORT UNDOABLE OPERATIONS.”*

COMMAND PATTERN

- ▶ *Encapsulate a request as an object, thereby letting users parameterize clients with different requests, queue or log requests, and support undoable operations.*
- ▶ Det var da en horribel beskrivelse...
 - ▶ Virker KUN når vi taler computer analogier, ellers er en klient vel et menneske, som ikke kan parametriseres ?
 - ▶ Resten er bare steder dette måske kan bruges...
- ▶ Det, det egentlig prøvet at beskrive er at command pattern'et er et objektificeret metode kald.
 - ▶ Dvs. Et metode kald wrapped ind i et objekt ☺

COMMAND PATTERN

- ▶ Client er ansvarlig for at oprette en konkret command og associere den med sin receiver.
- ▶ Invoker er ansvarlig for at kalde command'en
- ▶ Command er et interface eller abstract klasse, der indeholder signaturen for de(n) konkrete commands.
- ▶ kan kombineres så f.eks invoker og client er samme klasse.



COMMAND PATTERN - Hvorfor?

- ▶ **Fleksibilitet**
 - ▶ Nemmere at tilføje/fjerne commands uden at skulle ændre mange steder i systemet.
- ▶ **Undo/redo**
 - ▶ Ved at gemme en liste af udførte commands, og implementere en “unexecute” kan der implementeres hurtig og effektiv undo/redo
- ▶ **Genbrugelighed**
 - ▶ Command objekterne er genbrugelige, da det kan mappes til flere forskellige invokers.

COMMAND PATTERN

Hvor kan vi bruge det?

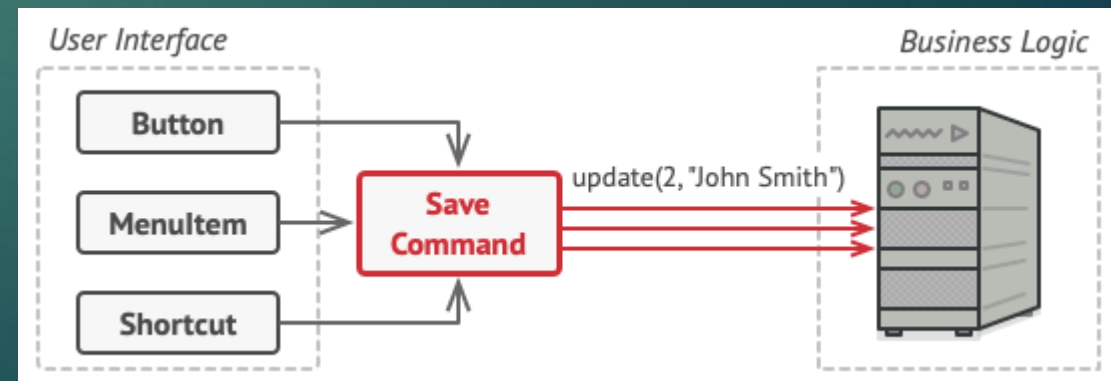
- ▶ Traditionel software:
 - ▶ Text editor
 - ▶ Indsæt, delete, copy
 - ▶ Kan både sættes som tastatur genvej, men også som museoperation
 - ▶ Lommeregner
- ▶ Alle operationer kan oprettes som commands.
- ▶ På dette måde har vi en nem måde at undo/redo
 - ▶ Kræver rækkefølgen er gemt.

```
public class PlusCommand : ICommand
{
    private int _lastAddedValue;
    private int _currentValue;

    0 references
    public PlusCommand(int currentValue)
    {
        _currentValue = currentValue;
    }

    1 reference
    public void Execute(int valueToAdd)
    {
        _lastAddedValue = valueToAdd;
        _currentValue += valueToAdd;
    }

    1 reference
    public void Undo()
    {
        _currentValue -= _lastAddedValue;
    }
}
```



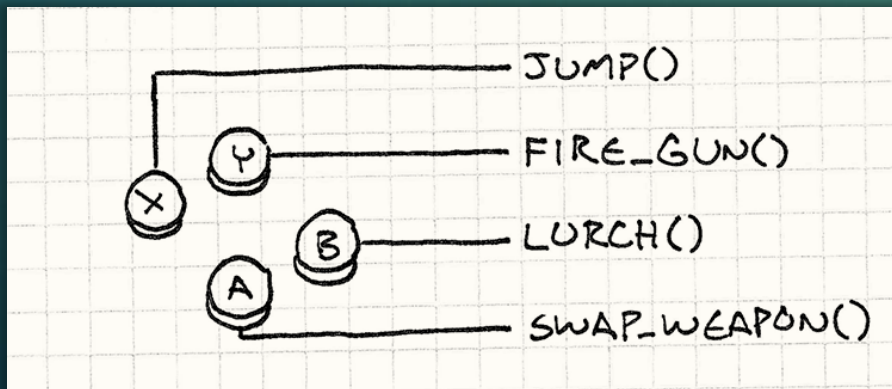
COMMAND PATTERN

Hvor kan vi bruge det?

- ▶ Spil
 - ▶ Bevægelse / bruger input
 - ▶ Kan mappe både til controller og til keyboard
 - ▶ Gør det også nemmere at rebind, ved at have alt input samlet samme sted
- ▶ User interface / menuer
 - ▶ Igen nemt at have sin funktionalitet adskilt fra selve GUI'en
 - ▶ Samme command kan udføres flere steder.
- ▶ Købe opgraderinger / powerups
 - ▶ Igen er undo/redo meget handy at have her!

COMMAND PATTERN eksempel

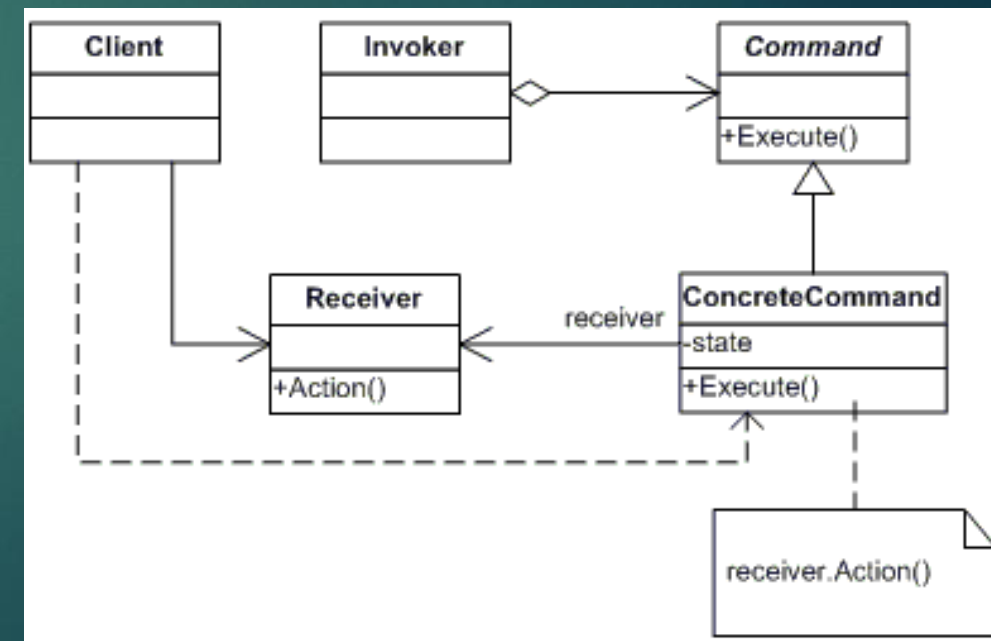
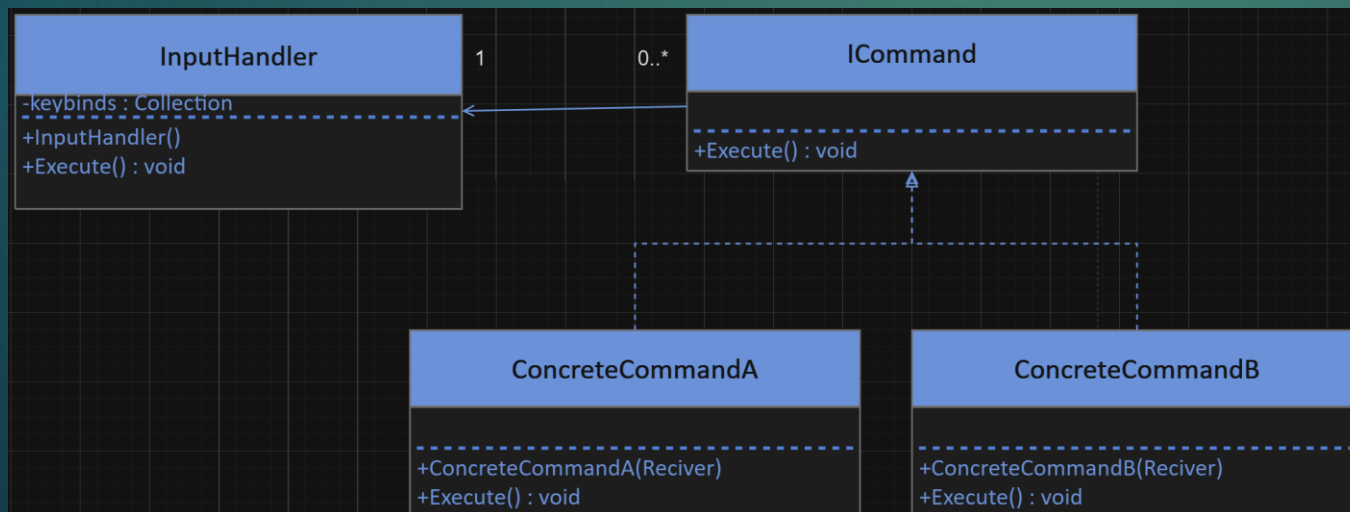
- ▶ Organisering af input commands
 - ▶ Gør det lettere at give brugeren mulighed for at udskifte keybinds i spillet.
 - ▶ Gør det muligt at samme funktionalitet kan kaldes fra forskellige klasser!
 - ▶ Vi gør ofte følgende:



```
if (keyState.IsKeyDown(Keys.A))  
{  
    SwapWeapon();  
}  
if (keyState.IsKeyDown(Keys.Y))  
{  
    FireGun();  
}  
if (keyState.IsKeyDown(Keys.B))  
{  
    Lurch();  
}  
if (keyState.IsKeyDown(Keys.X))  
{  
    Jump();  
}
```

COMMAND PATTERN

- ▶ Vi vil gerne ændre strukturen så det bliver lettere for os at ændre keybinds løbende.
 - ▶ Strukturen på en command pattern ser ud på følgende måde
 - ▶ Her er inputhandler både invoker og client



COMMAND PATTERN

▶ ICommand

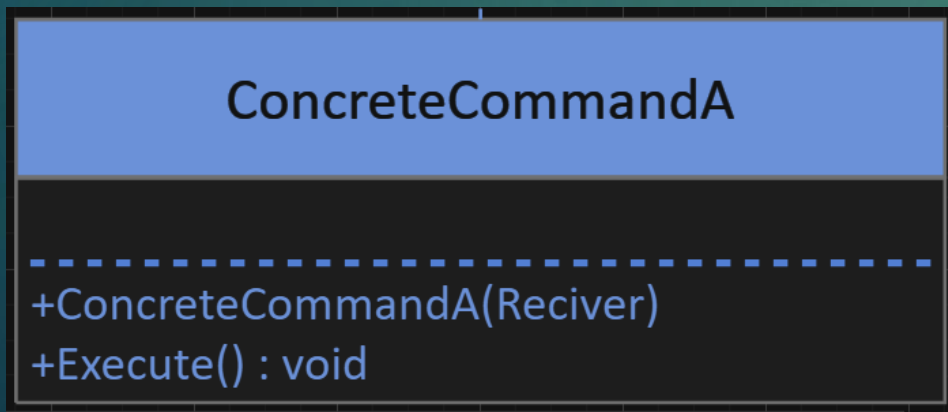
- ▶ Et interface eller abstract klasse, som definere strukturen af en command
- ▶ Execute udfører kommandoen på en entity vi styrer, f.eks. Vores player.
- ▶ Kan laves på flere måder
 - ▶ Entity (reciever) bør blive sat i constructoren constructor, men kan også sættes som del af execute.
 - ▶ Execute bør enten tage kun reciever eller ingenting som parameter.
 - ▶ Andre parametre skal vævre sat i Constructor.



COMMAND PATTERN

- ▶ ConcreteCommand

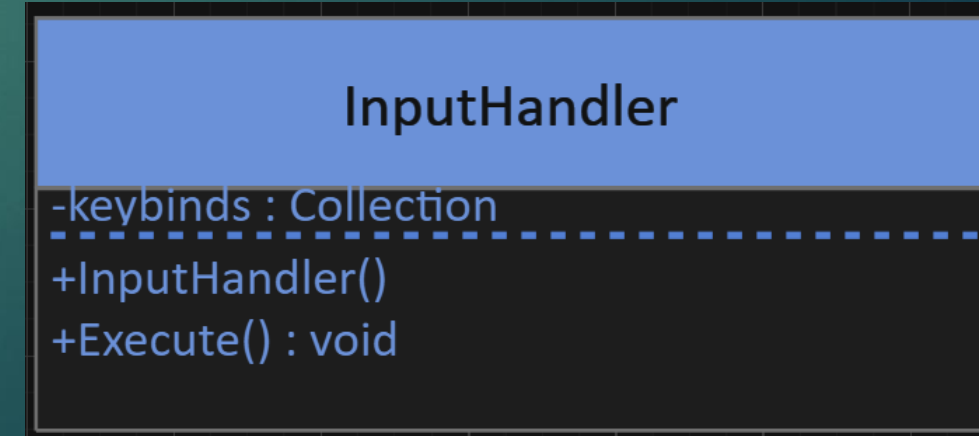
- ▶ En konkret implementering af en kommando. Kunne bl.a være en "MoveCommand", som får vores spiller til at flytte sig
 - ▶ Står nu et overfår spørgsmål – skal forskellige retninger være forskellige commands eller Parametre?



COMMAND PATTERN

▶ InputHandler

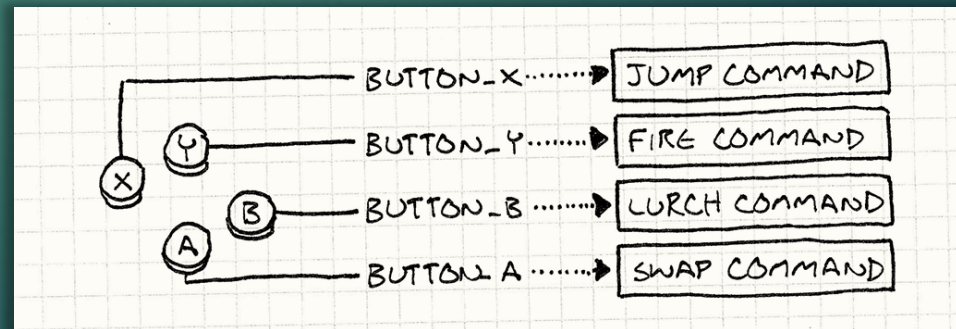
- ▶ Håndterer vores input
- ▶ Keybinds er en form for collection, som indeholder alle vores keybinds.
- ▶ Execute kører kommandoer, som er tilknyttet specifikke keybinds.
 - ▶ F.eks. D eksekverer MoveRight kommando.
- ▶ Den undersøger hvilket input der er trykket ned i "update" og kalder commanden der er mappet til denne knap, hvis nogen.



COMMAND PATTERN

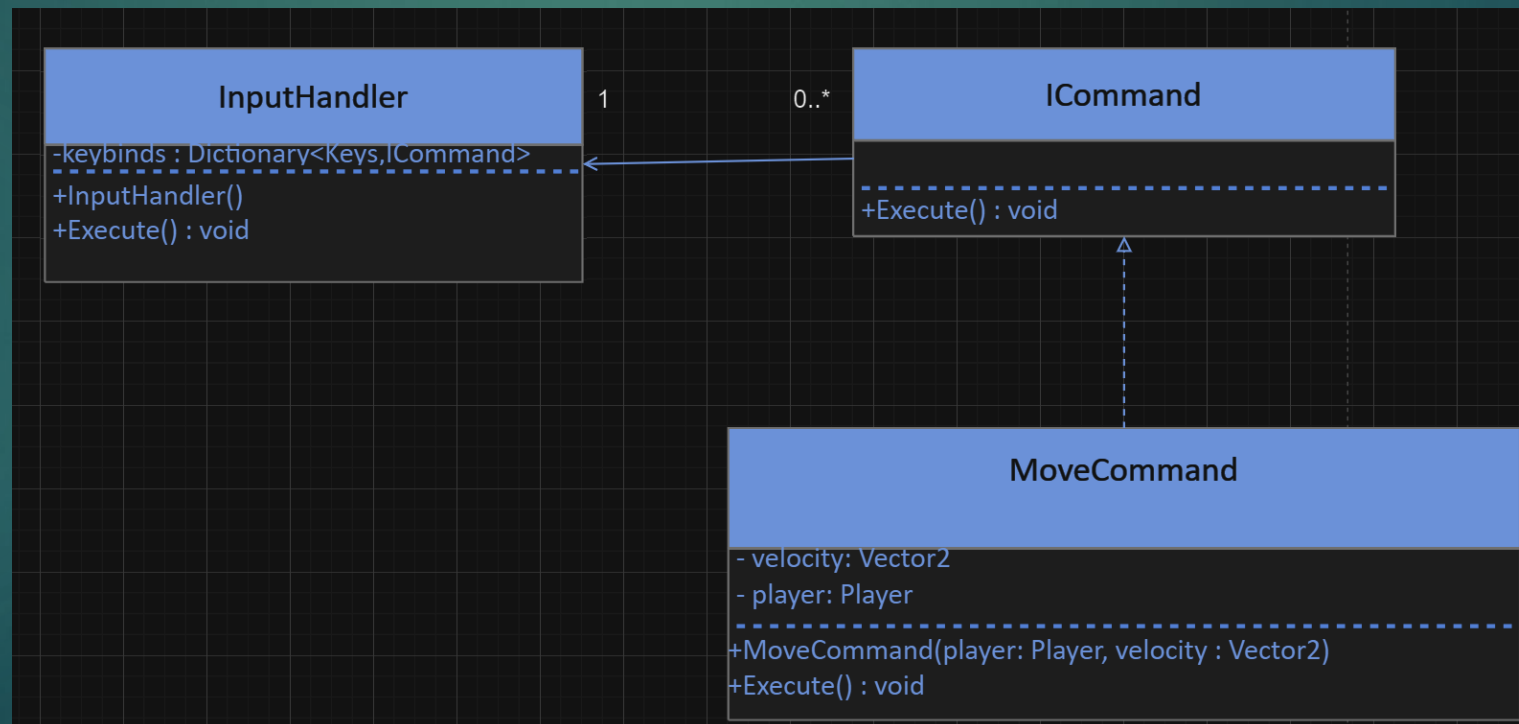
- ▶ Strukturen på koden skulle gerne se ud på følgende måde efter command pattern er blevet implementeret.
 - ▶ De mange if sætninger erstattes af en simpel løkke
 - ▶ Der er tilføjet et ekstra lag mellem vores inputs og vores actions.
 - ▶ I dette tilfælde er keybinds dictionary<keys, ICommand>

```
foreach (Keys key in keybinds.Keys)
{
    if (keyState.IsKeyDown(key))
    {
        keybinds[key].Execute(player);
    }
}
```



COMMAND PATTERN

- En konkret implementering i vores spil kunne se ud på følgende måde



COMMAND PATTERN

- ▶ ICommand
 - ▶ Et interface, som bestemmer strukturen af alle vores commands



```
interface ICommand
{
    2 references
    void Execute();
}
```

COMMAND PATTERN

- ▶ MoveCommand
 - ▶ Får vores spiller til at flytte sig.
 - ▶ Bemærk at der er tilføjet to fields og en constructor.



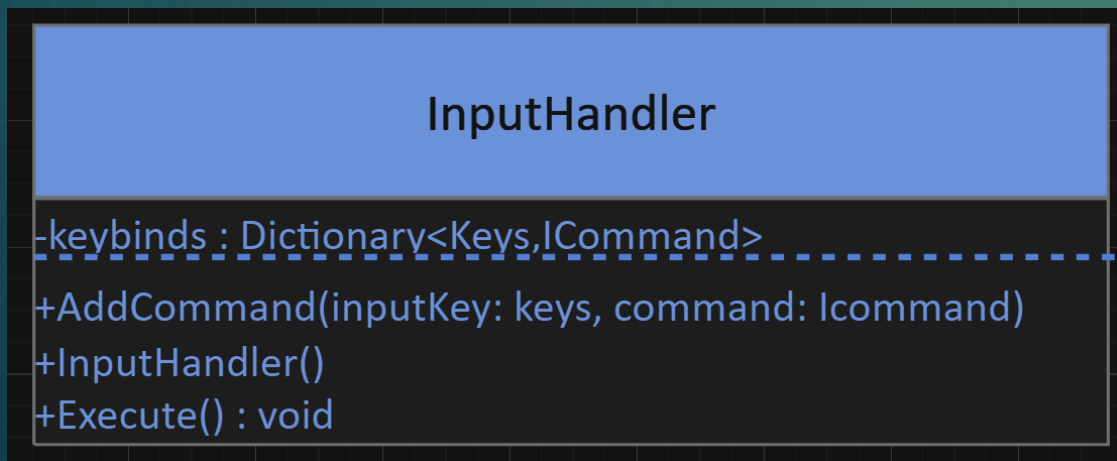
```
3 references
class MoveCommand : ICommand
{
    private Player player;
    private Vector2 velocity;

    2 references
    public MoveCommand(Player player, Vector2 velocity)
    {
        this.player = player;
        this.velocity = velocity;
    }

    2 references
    public void Execute()
    {
        player.Move(velocity);
    }
}
```

COMMAND PATTERN

- ▶ InputHandler
 - ▶ Opretter keybinds og kontrollerer om spilleren trykker på dem



```
4 references
class InputHandler
{
    private Dictionary<Keys, ICommand> keybinds = new Dictionary<Keys, ICommand>();

    2 references
    public void AddCommand(Keys inputKey, ICommand command)
    {
        keybinds.Add(inputKey, command);
    }

    1 reference
    public void Execute()
    {
        KeyboardState keyState = Keyboard.GetState();

        foreach (var pressedKey in keyState.GetPressedKeys())
        {
            if (keybinds.TryGetValue(pressedKey, out ICommand cmd))
            {
                cmd.Execute();
            }
        }
    }
}
```

ØVELSE 2

Implementer et Command pattern

- ▶ Sørg for at spilleren kan flytte sig horisontalt
- ▶ Sørg for at spilleren spawner centreret i bunden af skærmen når spillet startes
- ▶ Sørg for at inputhandleren bliver oprettet i GameWorld klassen, og får indsat sine Commands!
- ▶ Det kan også være playeren selv der har InputHandleren.

Løsning

https://www.youtube.com/watch?v=_kO-PdyHg-s

HINTS

- ▶ Følgende tilføjelser bør laves på jeres player
 - ▶ Move **kan** se ud på følgende måde:
 - ▶ **DeltaTime** kan beregnes i GameWorld Evt via. En static property.
 - ▶ Husk at ændre **Draw** så den gør brug af origin og position
 - ▶ **Execute** på InputHandler kan kaldes fra Player eller GameWorld Update

```
public void Move(Vector2 velocity)
{
    if (velocity != Vector2.Zero)
    {
        velocity.Normalize();
    }

    velocity *= speed;

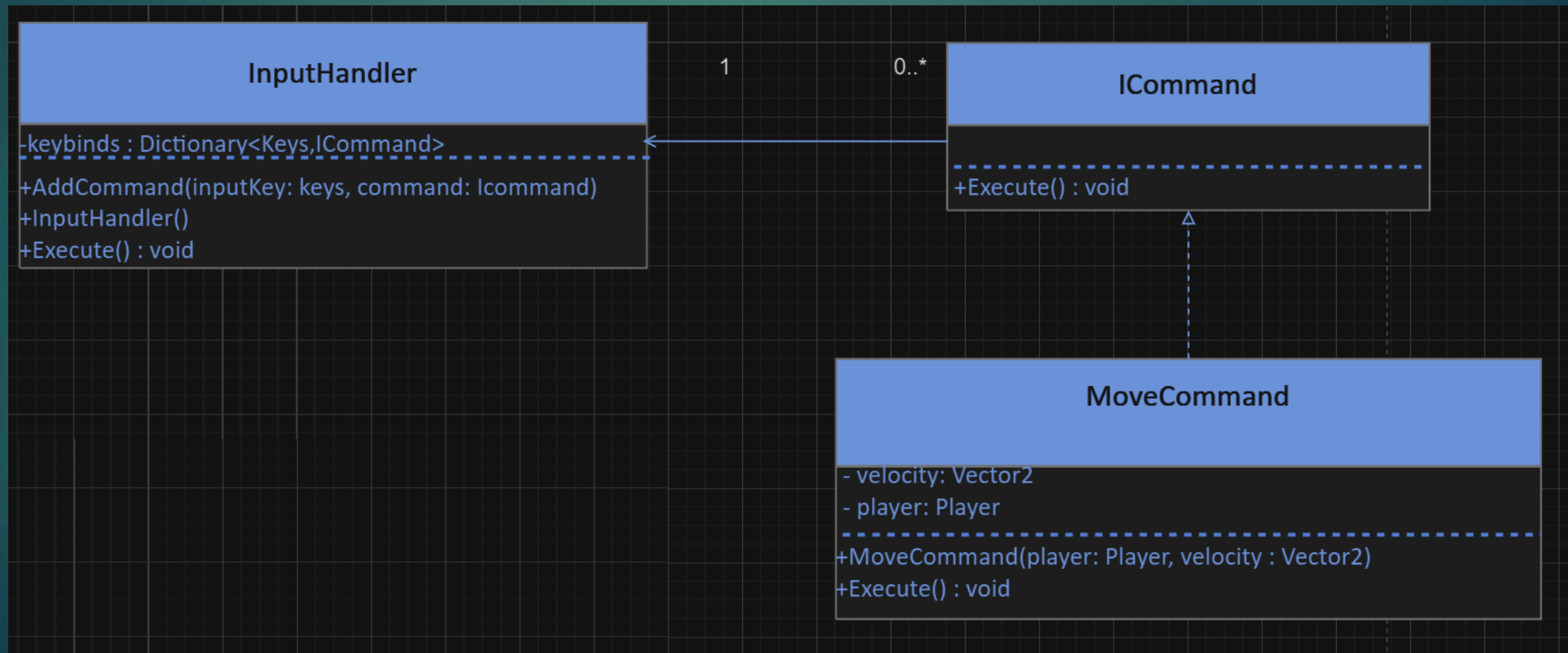
    position += (velocity * deltaTime);
}
```

Player
-sprite : Texture2D -speed : float -position : Vector2 -origin : Vector2 -inputHandler : InputHandler
+Player(startPosition : Vector2) +Move(velocity : Vector2) : void +LoadContent(content : ContentManager) : void +Update() : void +Draw(spriteBatch : SpriteBatch) : void

```
DeltaTime = (float)gameTime.ElapsedGameTime.TotalSeconds;
```

HINTS

► UML diagram



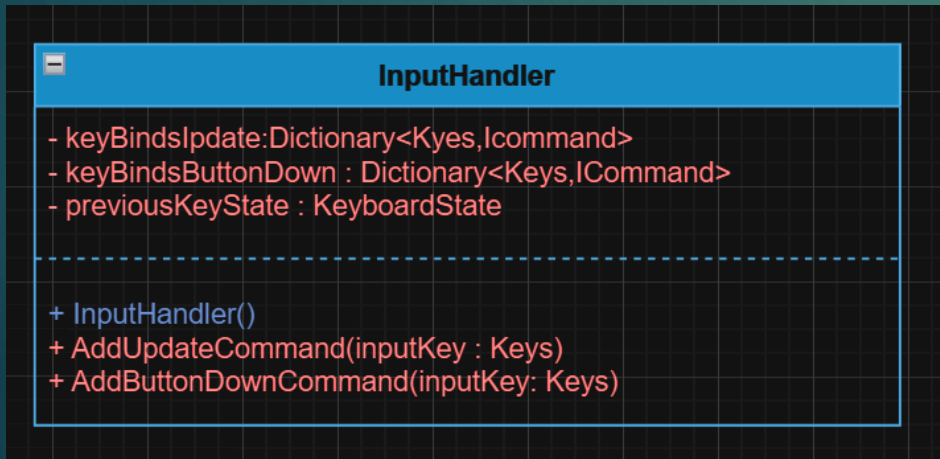
ØVELSE 3

- ▶ Udvid jeres implementation til at der er 2 forskellige typer af commands der kan eksekveres.
 - ▶ 1. den i har lavet nu, hvor commanden bliver kaldt hver frame
 - ▶ 2. Den nye, bliver Commanden kun bliver kaldt Hvis den ikke var kaldt sidste frame
 - ▶ F.eks(InputHandler):

```
private KeyboardState previousKeyState;  
1 reference  
public void Execute()  
{  
    KeyboardState keyState = Keyboard.GetState();  
  
    foreach (var pressedKey in keyState.GetPressedKeys())  
    {  
        if (keybindsUpdate.TryGetValue(pressedKey, out ICommand cmd))  
        {  
            cmd.Execute();  
        }  
        if (!previousKeyState.IsKeyDown(pressedKey) && keyState.IsKeyDown(pressedKey))  
        {  
            if (keybindsButtonDown.TryGetValue(pressedKey, out ICommand cmdBd))  
            {  
                cmdBd.Execute();  
            }  
        }  
    }  
    previousKeyState = keyState;  
}
```

ØVELSE 3

- ▶ Dette kræver selvfølgelig en opdatering af InputHandler:



```
private Dictionary<Keys, ICommand> keybindsUpdate = new Dictionary<Keys, ICommand>();
private Dictionary<Keys, ICommand> keybindsButtonDown = new Dictionary<Keys, ICommand>();
private KeyboardState previousKeyState;
2 references
public void AddUpdateCommand(Keys inputKey, ICommand command)
{
    keybindsUpdate.Add(inputKey, command);
}

4 references
public void AddButtonDownCommand(Keys inputKey, ICommand command)
{
    keybindsButtonDown.Add(inputKey, command);
}
```

ØVELSE 3

- ▶ Ville den eksisterende Move command blive brugt ,ville spilleren ikke bevæge sig særlig meget.
- ▶ Derfor kan en ny Command tilføjes. F.eks:



```
1 reference
public void MoveByAddition(Vector2 velocity)
{
    position += velocity;
}
```

Udvidelsen af player.cs

```
5 references
class TeleportCommand : ICommand
{
    private Player player;
    private Vector2 velocity;

    4 references
    public TeleportCommand(Player player, Vector2 velocity)
    {
        this.player = player;
        this.velocity = velocity;
    }

    4 references
    public void Execute()
    {
        player.MoveByAddition(velocity);
    }
}
```

ØVELSE 3

- ▶ Husk and bind de nye commands!
 - ▶ ...og giv dem store nok værdier til at de kan se de gør noget.

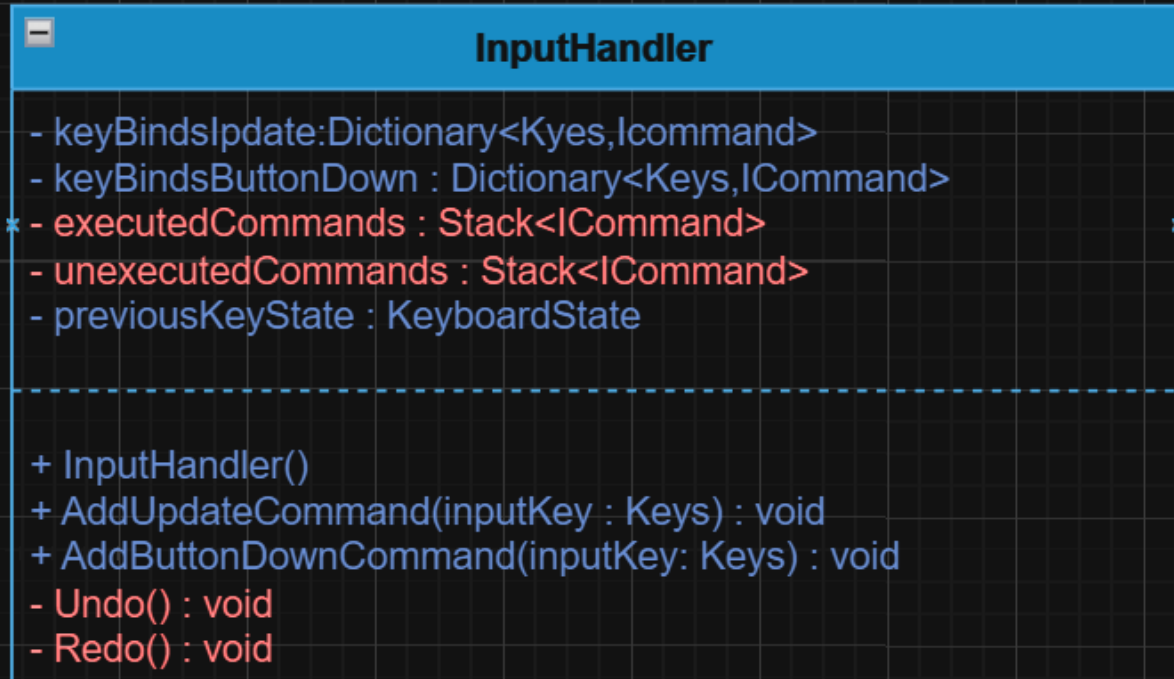
```
inputHandler.AddUpdateCommand(Keys.A, new MoveCommand(player, new Vector2(-1, 0)));  
inputHandler.AddButtonDownCommand(Keys.Q, new TeleportCommand(player, new Vector2(-1,-1)*10));  
inputHandler.AddButtonDownCommand(Keys.E, new TeleportCommand(player, new Vector2(1, -1) * 10));  
inputHandler.AddButtonDownCommand(Keys.Z, new TeleportCommand(player, new Vector2(-1, 1) * 10));  
inputHandler.AddButtonDownCommand(Keys.C, new TeleportCommand(player, new Vector2(1,1) * 10));
```

ØVELSE 4

Løsning

<https://www.youtube.com/watch?v=8xxhuTJOe7E>

- Udvid endnu mere til at have Undo og Redo funktionalitet.



```
if (!previousKeyState.IsKeyDown(pressedKey) && keyState.IsKeyDown(pressedKey))
{
    if (keybindsButtonDown.TryGetValue(pressedKey, out ICommand cmdBd))
    {
        cmdBd.Execute();
        executedCommands.Push(cmdBd);
        undoneCommands.Clear();
    }

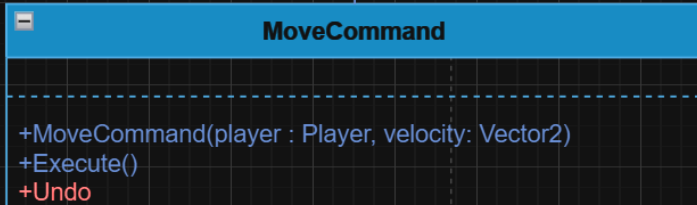
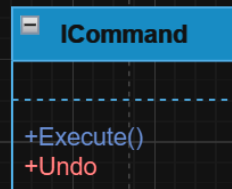
    if (pressedKey == Keys.O)
    {
        Redo();
    }
    if (pressedKey == Keys.P)
    {
        Undo();
    }
}

1 reference
private void Undo()
{
    if (executedCommands.Count > 0)
    {
        ICommand commandToUndo = executedCommands.Pop();
        commandToUndo.Undo();
        undoneCommands.Push(commandToUndo);
    }
}

1 reference
public void Redo()
{
    if (undoneCommands.Count > 0)
    {
        ICommand commandToRedo = undoneCommands.Pop();
        commandToRedo.Execute();
        executedCommands.Push(commandToRedo);
    }
}
```

ØVELSE 4

- Selvfølgelig skal ICommand også opdateres til at have en Undo



```
5 references
class TeleportCommand : ICommand
{
    private Player player;
    private Vector2 velocity;

    4 references
    public TeleportCommand(Player player, Vector2 velocity)
    {
        this.player = player;
        this.velocity = velocity;
    }

    4 references
    public void Execute()
    {
        player.MoveByAddition(velocity);
    }

    2 references
    public void Undo()
    {
        player.MoveByAddition(-velocity);
    }
}
```



SINGLETON PATTERN

Undervisning
<https://www.youtube.com/watch?v=SIVwePiVqlc>

SINGLETON

- ▶ Formål:
 - ▶ Sikre at en klasse kun har en instans og giver et globalt "point of access" til klasen
- ▶ Bruges f.eks.
 - ▶ Hvis man har brug for en HighscoreWriter klasse
 - ▶ Til en netværksforbindelse
 - ▶ Til en GameWorld
 - ▶ Til en GameManager



SINGLETON

- ▶ Hvorfor bruge singleton over static?
 - ▶ Kan implementere interfaces og nedarve fra andre klasser
 - ▶ Stiller et objekt til rådighed
 - ▶ Kan bruges i sammenhæng med andre patterns. F.eks. Factory
 - ▶ Fjerner funktionaliteten fra det globale namespace



SINGLETON

► UML og Kode

```
public class Player
{
    private static Player instance;
}
```

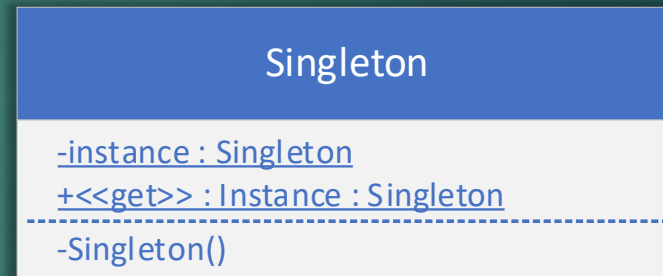


SINGLETON

► UML og Kode

```
public static Player Instance
{
    get
    {
        if (instance == null)
        {
            instance = new Player();
        }

        return instance;
    }
}
```



SINGLETON

- ▶ UML og Kode
 - ▶ Bemærk **privat** constructor

```
private Player()  
{  
  :  
  :  
}
```



SINGLETON

► UML og Kode

```
public class Player
{
    private static Player instance;

    0 references
    public static Player Instance
    {
        get
        {
            if (instance == null)
            {
                instance = new Player();
            }

            return instance;
        }
    }

    1 reference
    private Player()
    {
    }
}
```

Singleton

```
-instance : Singleton
+<<get>> : Instance : Singleton
-----
-Singleton()
```

SINGLETON

- ▶ UML og Kode
 - ▶ En metode kan kaldes på følgende måde fra en vilkårlig klasse.

```
Player.Instance.Shoot();
```



ØVELSE 5

Løsning

<https://www.youtube.com/watch?v=15xjPwz1SwU>

- ▶ Gør jeres InputHandler til en singleton
 - ▶ **HUSK PRIVATE CONSTRUCTOR!**
- ▶ Gør jeres GameWorld til en singleton
 - ▶ **HUSK PRIVATE CONSTRUCTOR!**
- ▶ Lav de nødvendige ændringer for at gøre brug af de nye implementeringer.

TRANSFORM KLASSE

Undervisning
<https://www.youtube.com/watch?v=HPA-ONVmtVM>

TRANSFORM

- ▶ Transform er en klasse, som alle vores GameObjects **skal** have.
- ▶ Det er ikke et pattern, men en meget god idé i spiludvikling.
- ▶ Transform bestemmer hvor vores GameObject befinder sig i vores GameWorld.
- ▶ Transform bruges til at flytte vores GameObject
 - ▶ Position: En property som bestemmer positionen på GameObjectet
 - ▶ Translate: Bruges til at manipulere med positionen.

Transform

+<<get,set>> Position : Vector2

+Translate(translation : Vector2) : void

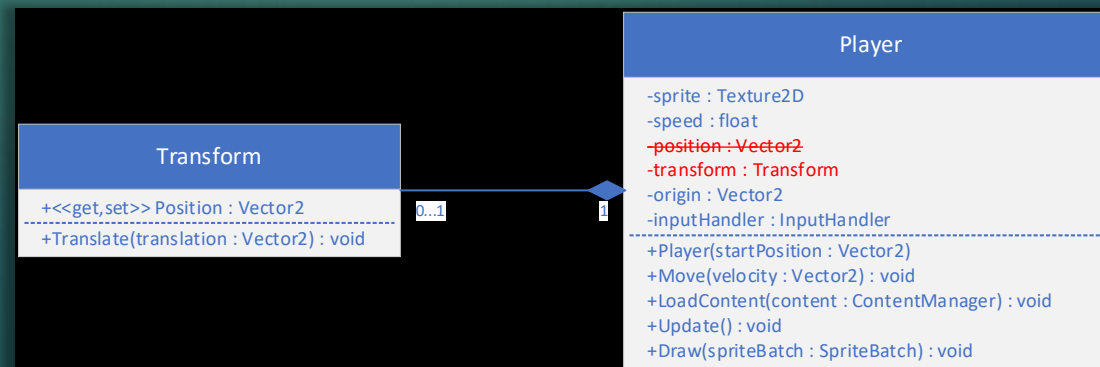
TRANSFORM

- Translate kan se ud på følgende måde:

```
public void Translate(Vector2 translation)
{
    Position += translation;
}
```

TRANSFORM

- ▶ Jeres Player skal gøre brug af den nye Transform klasse i stedet for sin position.
 - ▶ Slet position variabelen
 - ▶ Tilføj en Transform variabel (husk at instantiere den)
 - ▶ Sæt Transform's position i Constructoren
 - ▶ Benyt Transform's Position property i Draw.
 - ▶ Benyt Transform's Translate metode til at flytte spilleren i Move.



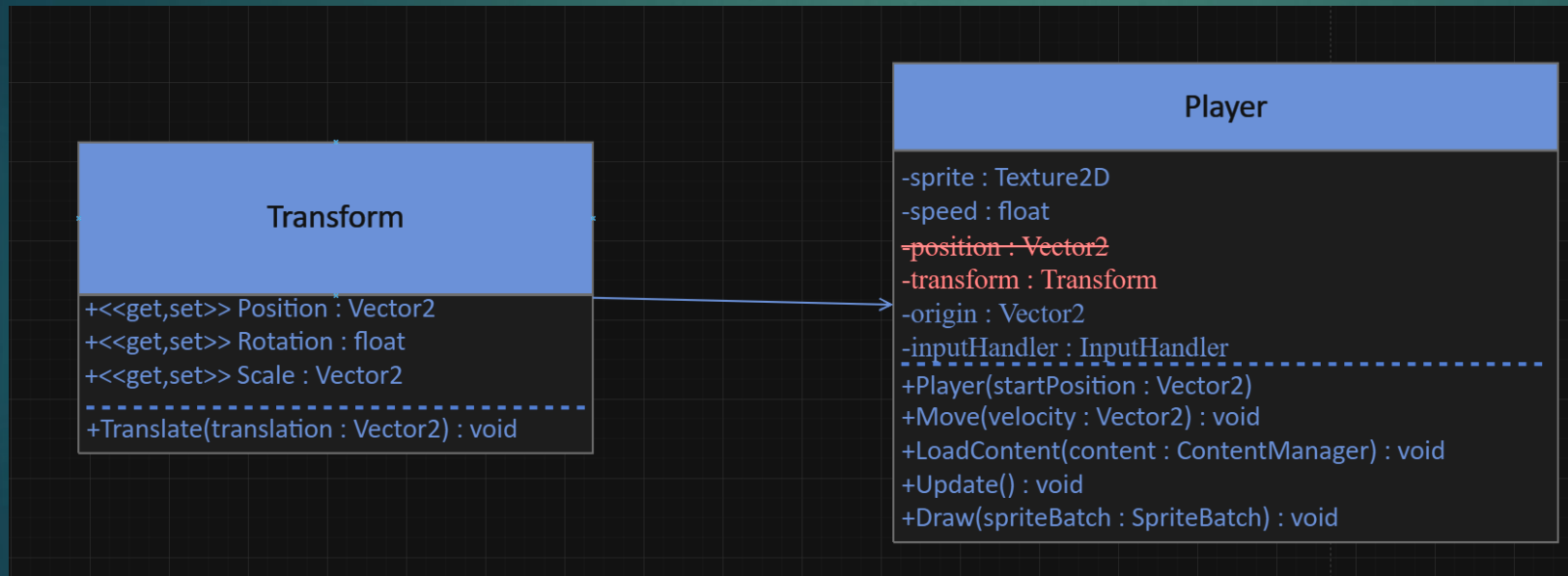
TRANSFORM

- ▶ Transform idéen kan bruges til mere end blot position, men alt der har noget at gøre med at være i en 2D verden
- ▶ Dvs. Rotation og skalering.
- ▶ Kigger vi på Draw overrides i Monogame kan vi se følgende:

```
spriteBatch.Draw()  
▲ 7 of 7 ▼ void SpriteBatch.Draw(Texture2D texture, Vector2 position, Rectangle? sourceRectangle, Color color, float rotation, Vector2 origin, Vector2 scale, SpriteEffects effects, float layerDepth)
```

TRANSFORM

- Med en smule udvidelse får vi derfor dette:



ØVELSE 6

Løsning

<https://www.youtube.com/watch?v=jgU0AhmVb28>

- ▶ Implementer Transform klassen i din Player.
 - ▶ Test at dine player stadig kan bevæge sig.
 - ▶ Vi har ikke brug for rotation, og scale til præcis denne case, men prøv stadig at implementer dem, og undersøg at de virker som de skal.

OBS: Rotation

- ▶ Rotation vil altid foregå rundt om "Origin", som er en del af Playeren, og IKKE transformen.
- ▶ Vær opmærksom på hvordan "1 fwd" sprite'en ser ud
- ▶ Origin er ved ilden, hvis har brugt standard udregning.



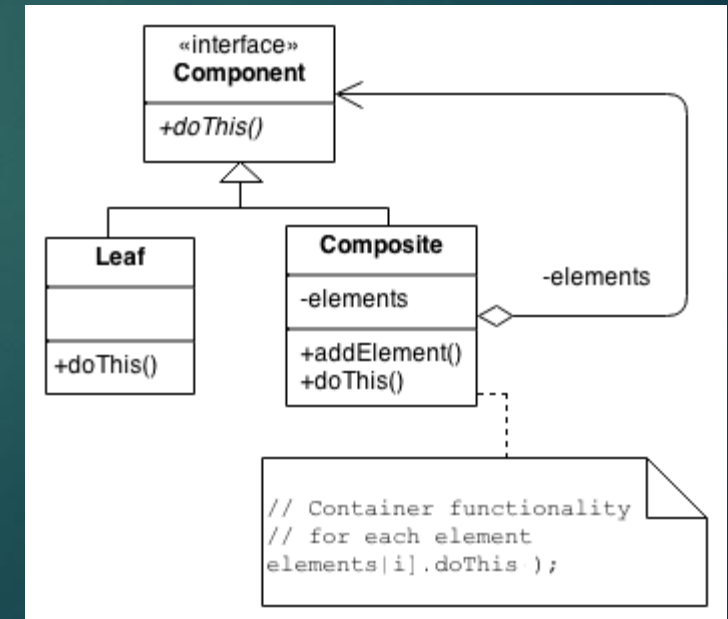


COMPONENT PATTERN

Undervisning
<https://www.youtube.com/watch?v=UVAHejTtOfA>

COMPONENT PATTERN

- ▶ Component pattern og også kendt som composite pattern.
- ▶ Formålet er at bruge en træ struktur fremfor nedrivning, for at undgå duplikeret kode.
- ▶ Gøres normalt ved at implementere et interface/abstrakt klasse med en "operation"
- ▶ Hvor der implementeres en "composite"
 - ▶ Denne composite kan have mange "leafs".



COMPONENT PATTERN - eksempel

```
0 references
public abstract class Component
{
    5 references
    public abstract void Operation();
}

// Leaf
```

```
1 reference
public class Leaf : Component
{
    4 references
    public override void Operation()
    {
        Console.WriteLine("Leaf operation");
    }
}
```

```
class Program
{
    0 references
    static void Main()
    {
        Component leaf = new Leaf();
        Composite composite = new Composite();
        composite.Add(leaf);

        // Kan kalde Operation på både leaf og composite på samme måde
        leaf.Operation();
        composite.Operation();
    }
}
```

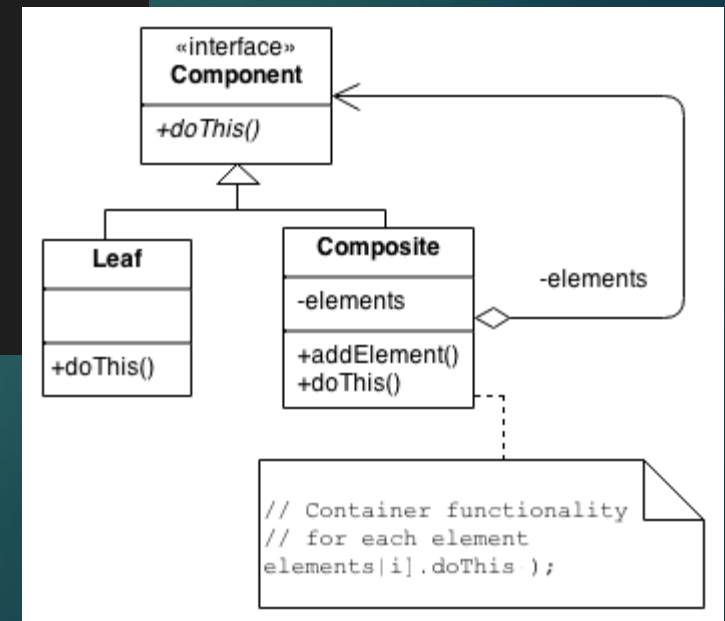
```
public class Composite : Component
{
    private List<Component> children = new List<Component>();

    0 references
    public void Add(Component component)
    {
        children.Add(component);
    }

    0 references
    public void Remove(Component component)
    {
        children.Remove(component);
    }

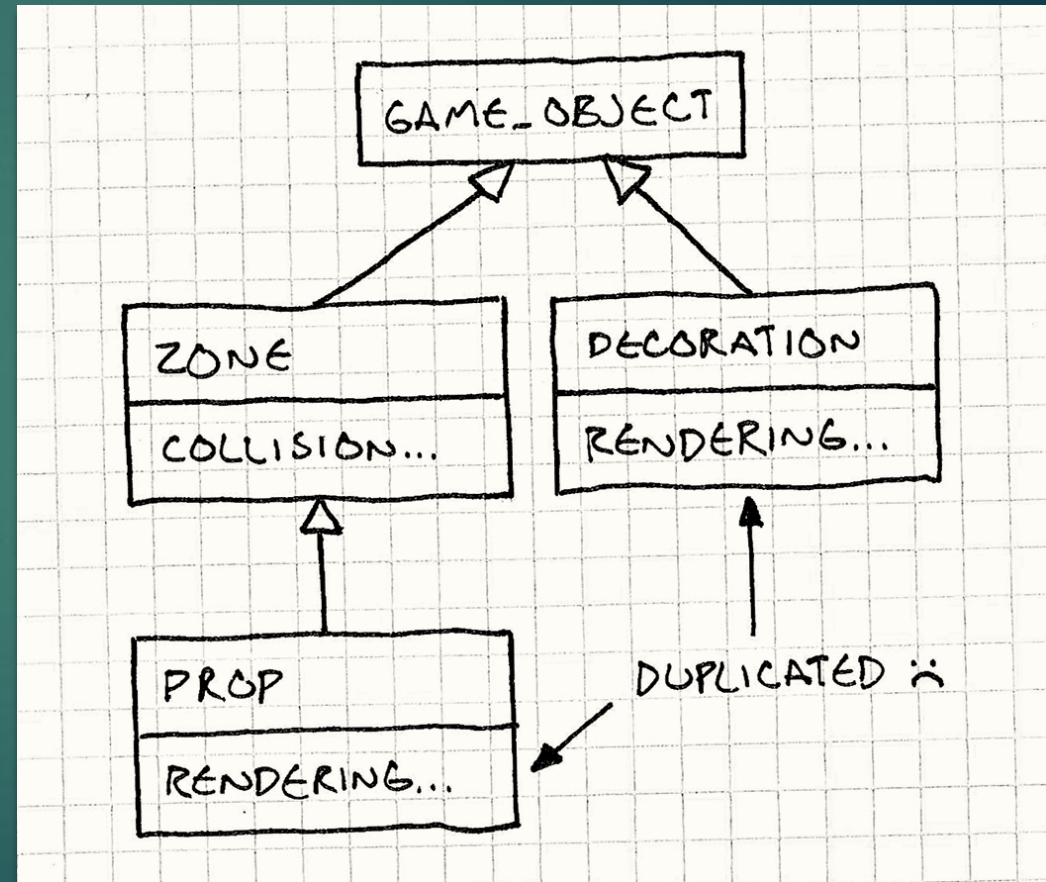
    4 references
    public override void Operation()
    {
        Console.WriteLine("Composite operation");
        foreach (var child in children)
        {
            child.Operation();
        }
    }
}
```

```
Leaf operation
Composite operation
Leaf operation
```



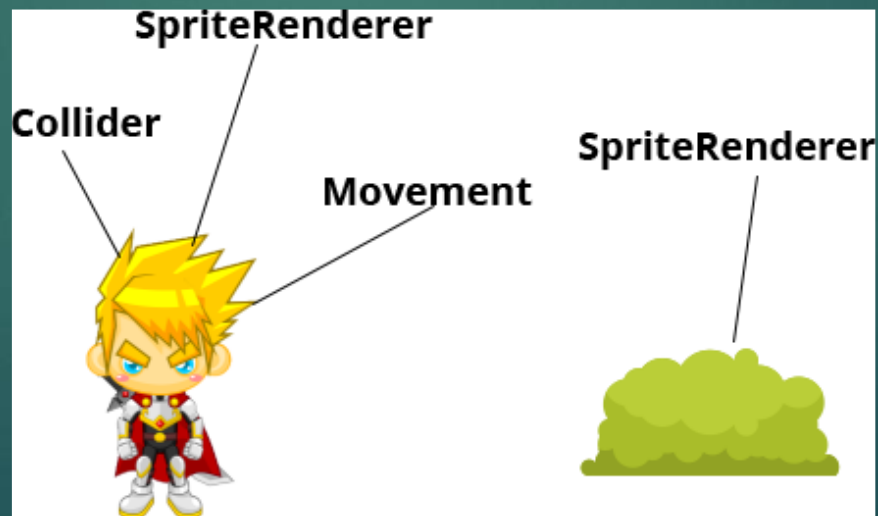
COMPONENT PATTERN

- ▶ Nedarving er en meget stor fordel i OOP men i komplekse nedarvingshierarkier kan man komme ud for:
 - ▶ At klasser implementerer funktionalitet, som de ikke gør brug af.
 - ▶ At den samme funktionalitet skal implementeres flere gange.



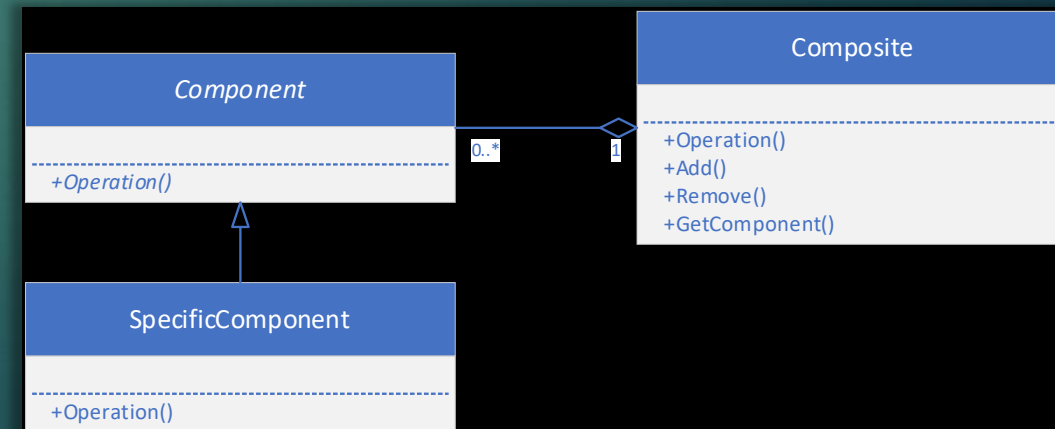
COMPONENT PATTERN

- ▶ Et component pattern kan løse disse problemer.
 - ▶ Giver os mulighed for at tilpasse vores objekter ved kun at tilføje de komponenter/Elementer de har brug for.



COMPONENT PATTERN

- ▶ Generel struktur af et Component pattern
 - ▶ Component: Superklassen til alle komponenter.
 - ▶ SpecificComponent: En specifikt komponent.
 - ▶ Composite: En klasse, som implementerer 0..* components
 - ▶ Add tilføjer et component, Remove fjerner et, GetComponent finder et specifikt komponent.
 - ▶ OBS: Composite er IKKE en component selv her.
 - ▶ Dvs uden "leafs"



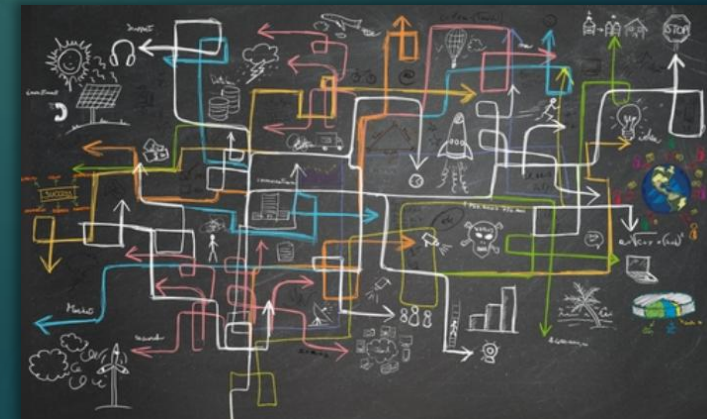
COMPONENT PATTERN

► Fordele

- Gør det muligt at isolere kode, så man undgår store og uoverskuelige klasser.
 - F.eks. Koden i et GameObject bliver hurtig uoverskuelig og indeholder ofte mange metoder (rendering, kollision, movement osv.)
 - Koden kan deles op i komponenter, som i stedet tager sig af logikken.
 - Genbruglige!

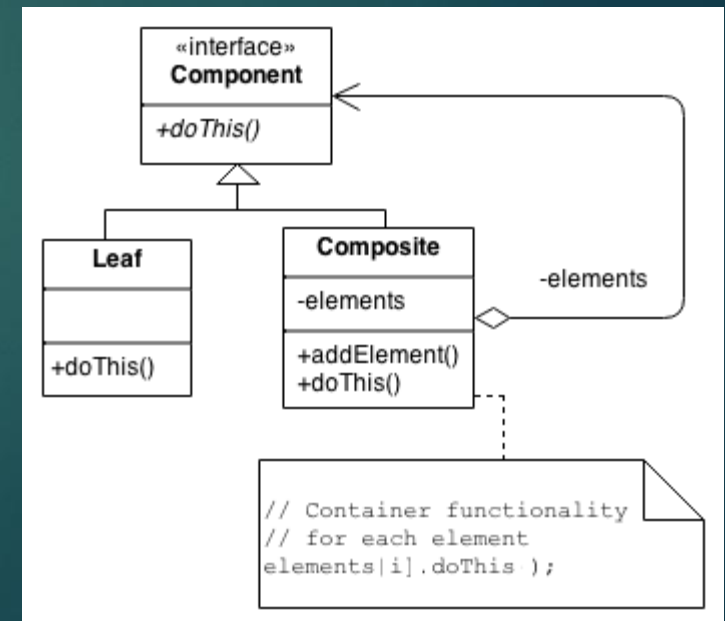
► Ulemper

- Nogle ting bliver mere komplekse.
- Man skal skrive mere kode for at tilføje et enkelt objekt



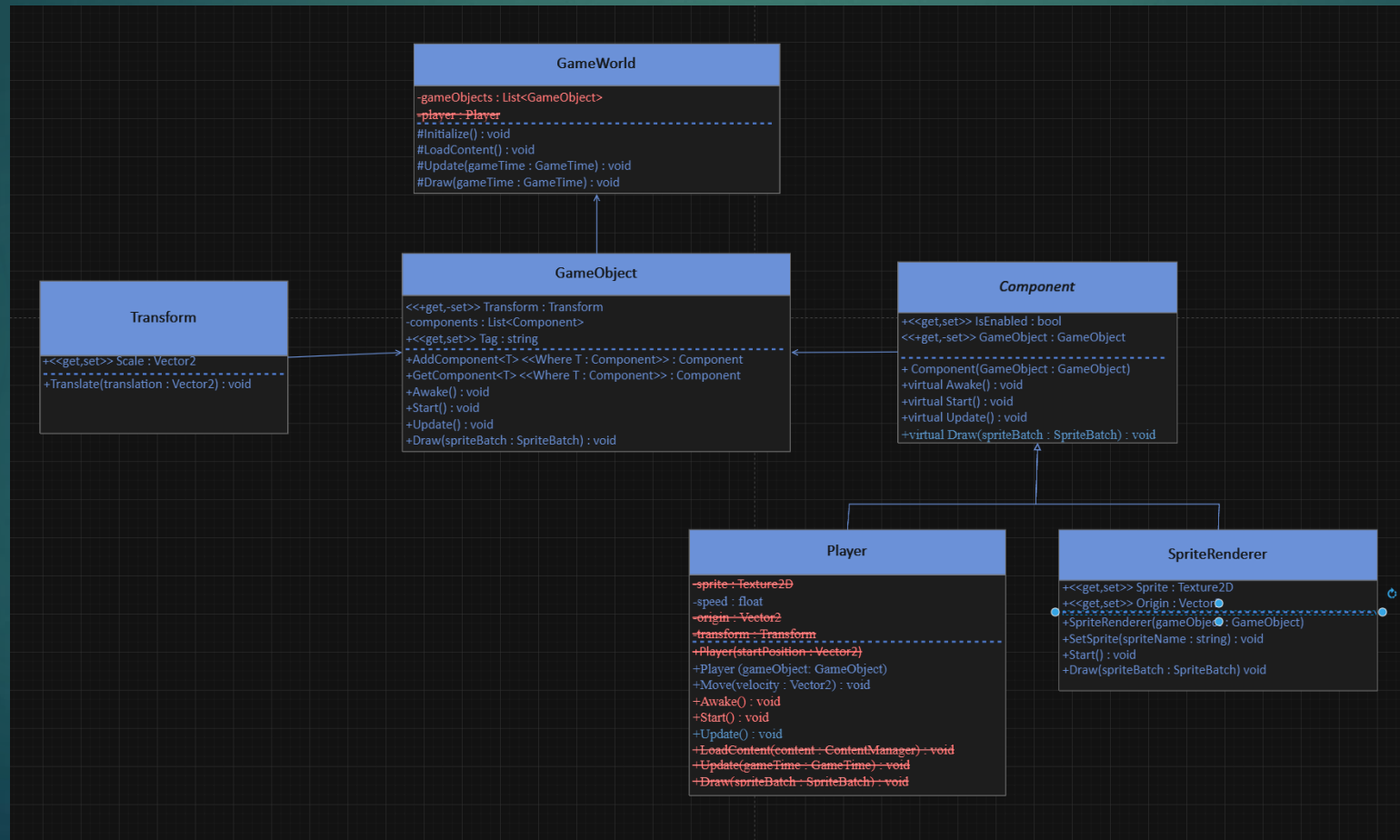
KONVERTERING

- ▶ Vi skal i gang med at konvertere vores projekt, så det gør brug af et Component pattern.
- ▶ For at vi kan gøre det skal følgende ske:
 1. Vi skal oprette en **Component** klasse (**abstract Component**)
 2. Vi skal oprette en **GameObject** klasse (**Composite**)
 3. Vi skal oprette en **SpriteRenderer** klasse (**leaf/specific component**)
 4. Vores **Player** skal laves til et **Component** (**leaf/specific component**)
 5. Der skal laves et par ændringer i **GameWorld** (**client**)



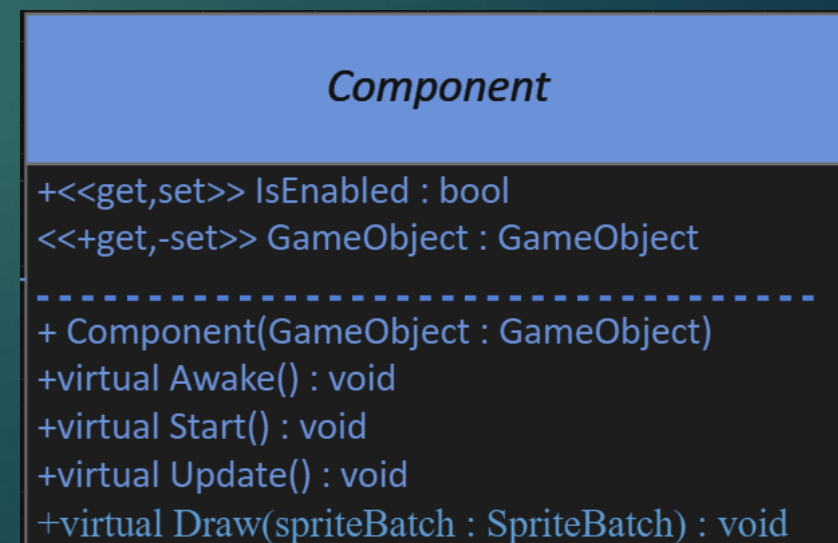
COMPONENT PATTERN

- Vores samlede pattern, kommer til at se ud på følgende måde:



COMPONENT PATTERN - Component

- ▶ Component klassen ser ud på følgende måde:
 - ▶ Bemærk at klassen er abstract.
 - ▶ En component kan ikke eksistere i sig selv.
 - ▶ `IsEnabled`: Property, som bestemmer om dette Component er aktivt.
 - ▶ Dvs. skal metoderne på objektet eksekveres.
 - ▶ `GameObject`: En reference til det `GameObject` dette Component tilhører.
 - ▶ Det vil også være præcis dette `gameObject` der opretter component'en!
 - ▶ Virtuelle metoder:
 - ▶ Tillader specifikke Components, at lave deres egen implementering af den givende metode.
 - ▶ `Awake`: Kaldes en gang per component
 - ▶ `Start`: Kaldes efter `Awake`, en gang per component
 - ▶ Kode som er afhængig af at `Awake` har kørt kan køres her
 - ▶ `Update` kaldes hver frame
 - ▶ `Draw` kaldes hver gang der tegnes.



GAME OBJECT KLASSENS FIELDS

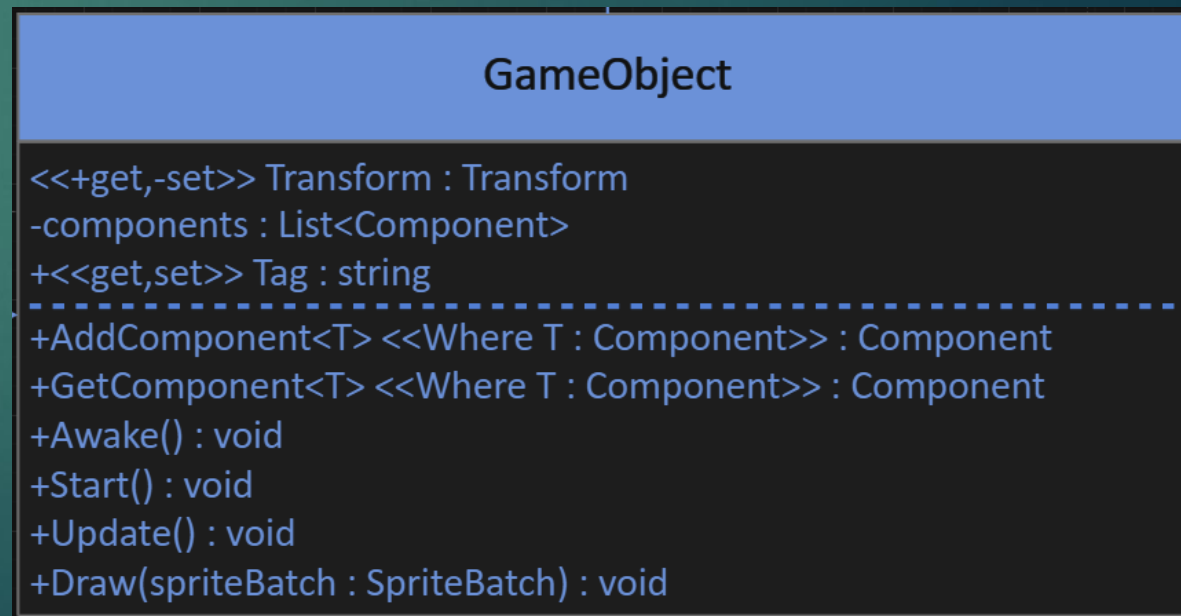
- ▶ Transform:
 - ▶ Alle GameObjects **skal** have en Transform. Dette skyldes at alle objekter skal have en position i spillet.
- ▶ Components:
 - ▶ En liste, som indeholder alle components der tilhører dette GameObject
- ▶ Tag:
 - ▶ Et tag er en string, som kan bruges til at genkende Objektet. Skal bruges senere.

GameObject
<<+get,-set>> Transform : Transform -components : List<Component> +<<get,set>> Tag : string

+AddComponent<T> <<Where T : Component>> : Component +GetComponent<T> <<Where T : Component>> : Component +Awake() : void +Start() : void +Update() : void +Draw(spriteBatch : SpriteBatch) : void

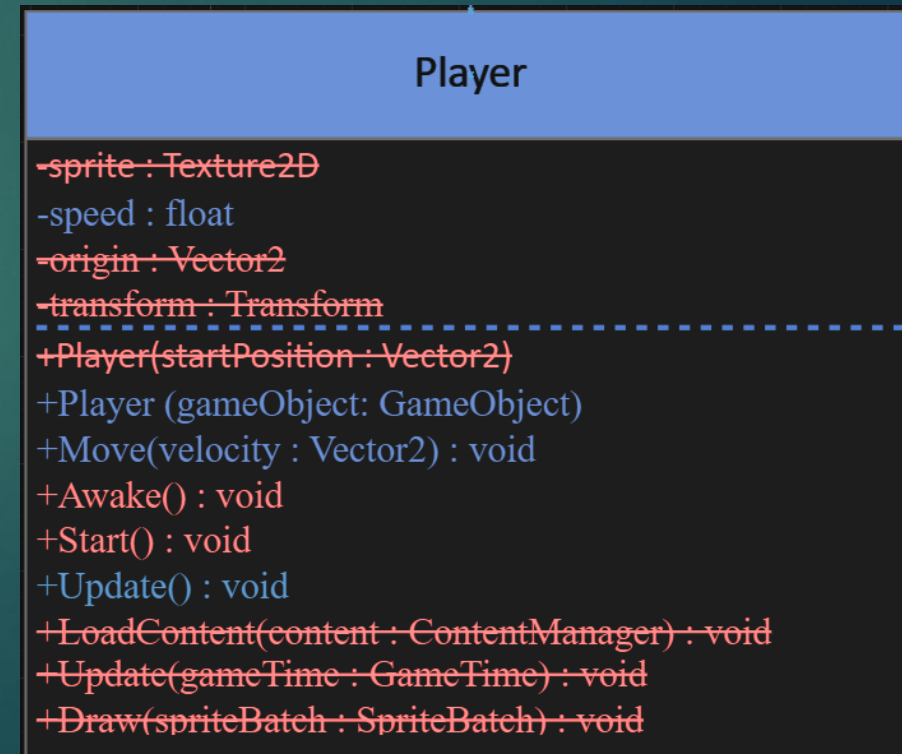
GAME OBJECT KLASSENS METODER

- ▶ AddComponent
 - ▶ Opretter og tilføjer en Component baseret på type.
- ▶ GetComponent
 - ▶ Returnerer et component baseret på en generisk værdi
- ▶ Awake
 - ▶ Kalder Awake på alle components
- ▶ Start
 - ▶ Kalder Start på alle components
- ▶ Draw
 - ▶ Kalder Draw på alle components



PLAYER

- ▶ Player skal nedarve fra component
 - ▶ Constructor Bruger signatur fra Component
- ▶ Sprite og transform fjernes fra vores Player
 - ▶ Transform ligger nu i GameObject
 - ▶ Sprite & origin ligger nu i SpriteRenderer (Næste slide)
- ▶ Draw og Load Content slettes
- ▶ Awake og Start tilføjes (alle sammen overrides)
 - ▶ Move: Fungerer som før
 - ▶ Start: Sætter spillerens start position (gennem `gameObject.transform`)
 - ▶ Start: Sætter spillerens sprite (gennem `spriteRenderer`)
 - ▶ Sprite renderer game fåes ved `gameObject.GetComponent`
 - ▶ Update: Kan overrides



SPRITERENDERER

- ▶ Nedarver fra Component
 - ▶ Sprite: En property, til den sprite, som komponenten skal tegne
 - ▶ Origin:
 - ▶ Spritsens origin
 - ▶ SetSprite:
 - ▶ Loader en sprite baseret på en string.
 - ▶ Start:
 - ▶ Origin kan sættes her.
 - ▶ Draw:
 - ▶ Tegner spriten

SpriteRenderer

```
+<<get,set>> Sprite : Texture2D
+<<get,set>> Origin : Vector2
-----
+SpriteRenderer(gameObject : GameObject)
+SetSprite(spriteName : string) : void
+Start() : void
+Draw(spriteBatch : SpriteBatch) void
```

GAMEWORLD Fields

- ▶ Diagrammet viser kun relevante members...
- ▶ GameObjects:
 - ▶ En liste som indeholder alle GameObjects
 - ▶ GameObjects bliver oprettet af GameWorld
- ▶ Player:
 - ▶ Fjernes og tilføjes til gameObjects listen som et GameObject.

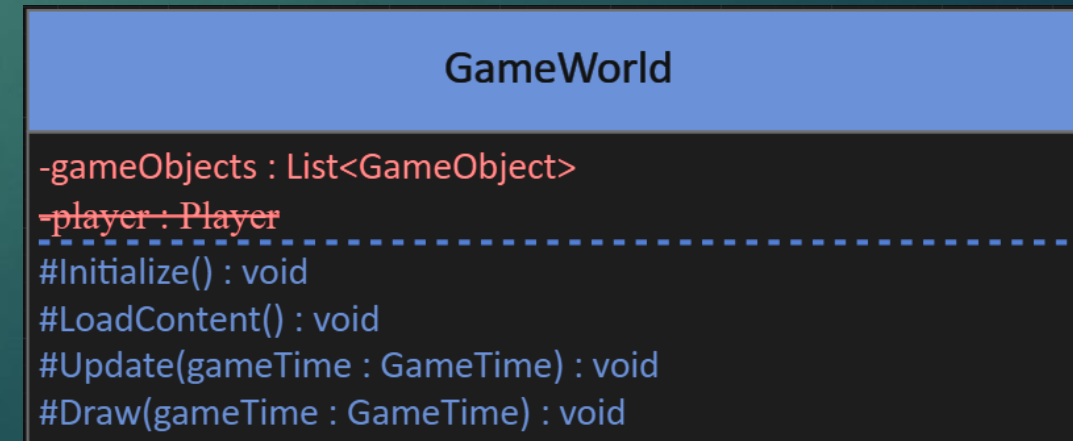


GAMEWORLD METODER

► Initialize

- Opretter alle GameObject's og tilføjer de nødvendige components.
- Kalder Awake på alle GameObjects, som kalder awake på alle dens components.

```
protected override void Initialize()
{
    // TODO: Add your initialization logic here
    GameObject playerGo = new GameObject();
    Player player = playerGo.AddComponent<Player>();
    playerGo.AddComponent<SpriteRenderer>();
    gameObjects.Add(playerGo);
    foreach (GameObject go in gameObjects)
    {
        go.Awake();
    }
}
```



GAMEWORLD METODER

► LoadContent

- Kalder Start på alle GameObjects, som kalder Start på alle dens components.

```
protected override void LoadContent()
{
    _spriteBatch = new SpriteBatch(GraphicsDevice);

    foreach (GameObject go in gameObjects)
    {
        go.Start();
    }
}
```

GameWorld

```
-gameObjects : List<GameObject>
-player : Player
-----
#Initialize() : void
#LoadContent() : void
#Update(gameTime : gameTime) : void
#Draw(gameTime : gameTime) : void
```

GAMEWORLD METODER

► Update

- Kalder Update på alle GameObjects , som kalder Update på alle dens components.

```
protected override void Update(GameTime gameTime)
{
    if (GamePad.GetState(PlayerIndex.One).Buttons.Back == ButtonState.Pressed || Keyboard.GetState().IsKeyDown(Keys.Escape))
        Exit();

    DeltaTime = (float)gameTime.ElapsedGameTime.TotalSeconds;
    foreach (GameObject go in gameObjects)
    {
        go.Update(gameTime);
    }
    InputHandler.Instance.Execute();
    base.Update(gameTime);
}
```

GameWorld

```
-gameObjects : List<GameObject>
-player : Player
-----
#Initialize() : void
#LoadContent() : void
#Update(gameTime : GameTime) : void
#Draw(gameTime : GameTime) : void
```

GAMEWORLD METODER

► Draw

- Kalder draw på alle GameObjects , som kalder Draw på alle dens components.

```
protected override void Draw(GameTime gameTime)
{
    GraphicsDevice.Clear(Color.CornflowerBlue);

    _spriteBatch.Begin();

    foreach (GameObject go in gameObjects)
    {
        go.Draw(_spriteBatch);
    }

    _spriteBatch.End();

    base.Draw(gameTime);
}
```

GameWorld

```
-gameObjects : List<GameObject>
-player : Player
-----
#Initialize() : void
#LoadContent() : void
#Update(gameTime : GameTime) : void
#Draw(gameTime : GameTime) : void
```

ØVELSE 7

Løsning

<https://www.youtube.com/watch?v=qKZGflh3fMc>

- ▶ Implementer et component pattern.
- ▶ Erstat jeres player i GameWorld med et GameObject med følgende komponenter:
- ▶ GameObject
 - ▶ Transform
 - ▶ SpriteRenderer
 - ▶ Player
- ▶ Lav en liste af GameObject's i GameWorld for at gøre det lettere at kalde Awake, Start, Update og Draw.

HINTS

► Eksempler på Get og Add Component (Del af GameObject)

```
public T AddComponent<T>() where T : Component
{
    Type componentType = typeof(T);
    // Get All constructors
    var constructors = componentType.GetConstructors();
    // Find Constructor with exactly 1 param that is a GameObject
    var constructor = constructors.FirstOrDefault(c =>
    {
        var parameters = c.GetParameters();
        return parameters.Length == 1 && parameters[0].ParameterType == typeof(GameObject);
    });
    if (constructor != null)
    {
        //Create instance of component using the Activator Class with This GameObject as A parameter.
        T component = (T)Activator.CreateInstance(componentType, this);
        components.Add(component);
        return component;
    }
    else
    {
        //Error handling...
        throw new InvalidOperationException($"Klassen {componentType.Name} skal have en konstruktør med ét parameter af typen GameObject.");
    }
}
```

HINTS

- ▶ Eksempler på Get og Add Component (Del af GameObject)

```
public Component GetComponent<T>() where T : Component
{
    return components.Find(x => x.GetType() == typeof(T));
}
```

1 reference